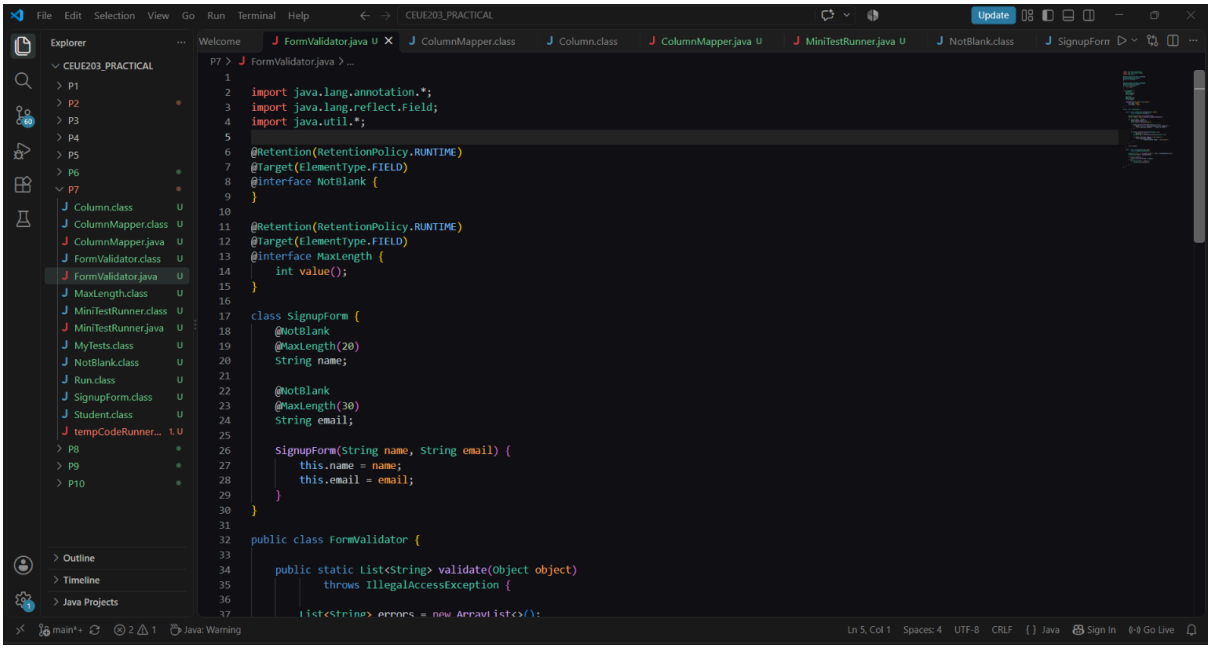
PRACTICAL 7

Form validator: define @NotBlank and @MaxLength(int) for fields and annotate a SignupForm. Write a checker that uses reflection to read each field's annotations and returns the list of validation errors for a given object.

CODE :



```java
import java.lang.annotation.*;
import java.lang.reflect.Field;
import java.util.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface NotBlank {
}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MaxLength {
    int value();
}

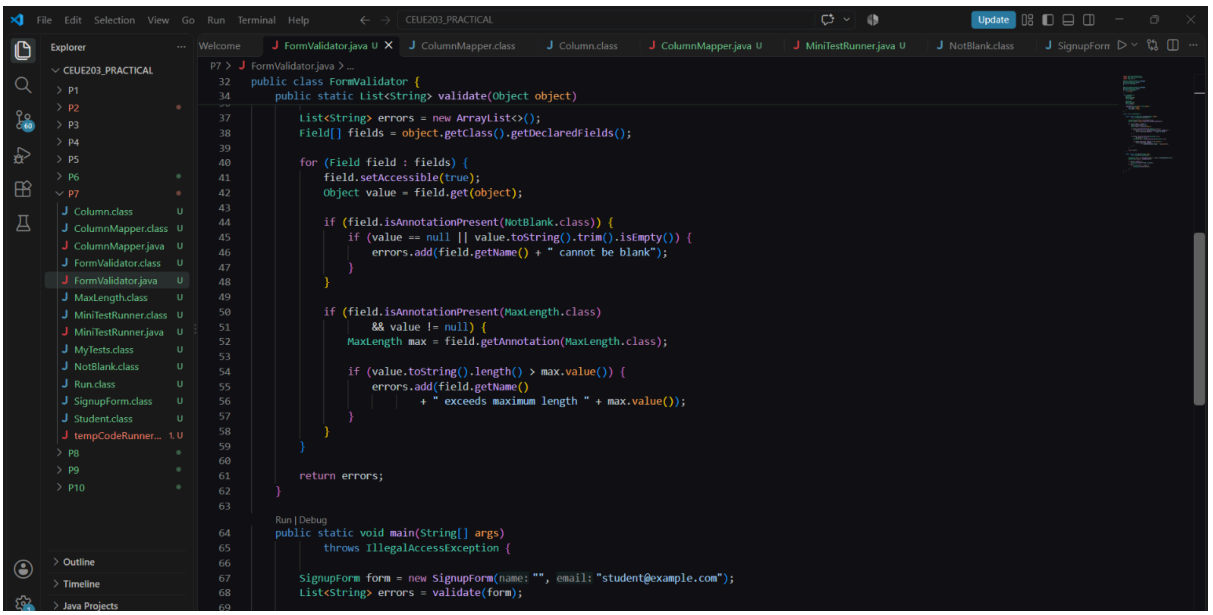
class SignupForm {
    @NotBlank
    @MaxLength(20)
    String name;

    @NotBlank
    @MaxLength(30)
    String email;

    SignupForm(String name, String email) {
        this.name = name;
        this.email = email;
    }
}

public class FormValidator {

    public static List<String> validate(Object object)
            throws IllegalAccessException {
```



```java
public class FormValidator {
    public static List<String> validate(Object object)

        List<String> errors = new ArrayList<>();
        Field[] fields = object.getClass().getDeclaredFields();

        for (Field field : fields) {
            field.setAccessible(true);
            Object value = field.get(object);

            if (field.isAnnotationPresent(NotBlank.class)) {
                if (value == null || value.toString().trim().isEmpty()) {
                    errors.add(field.getName() + " cannot be blank");
                }
            }

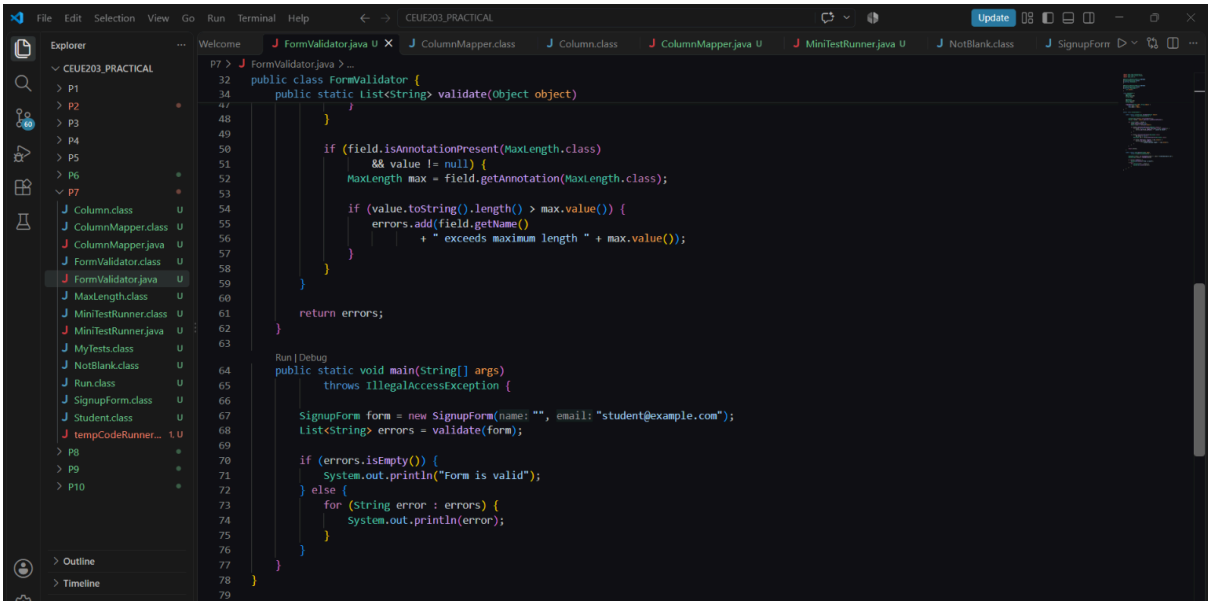
            if (field.isAnnotationPresent(MaxLength.class)
                    && value != null) {
                MaxLength max = field.getAnnotation(MaxLength.class);

                if (value.toString().length() > max.value()) {
                    errors.add(field.getName()
                            + " exceeds maximum length " + max.value());
                }
            }
        }

        return errors;
    }

    Run | Debug
    public static void main(String[] args)
            throws IllegalAccessException {

        SignupForm form = new SignupForm(name: "", email: "student@example.com");
        List<String> errors = validate(form);
```



```java
public class FormValidator {
    public static List<String> validate(Object object)
            }
        }

        if (field.isAnnotationPresent(MaxLength.class)
                && value != null) {
            MaxLength max = field.getAnnotation(MaxLength.class);

            if (value.toString().length() > max.value()) {
                errors.add(field.getName()
                        + " exceeds maximum length " + max.value());
            }
        }
    }

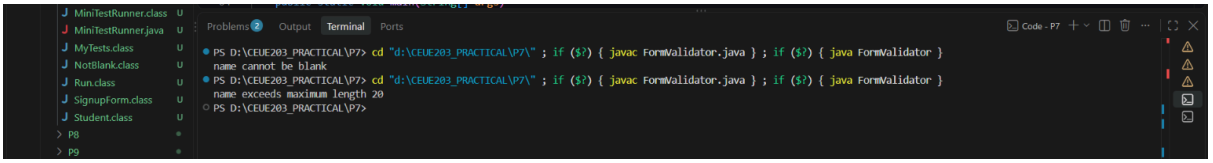
    return errors;
}

Run | Debug
public static void main(String[] args)
        throws IllegalAccessException {

    SignupForm form = new SignupForm(name: "", email: "student@example.com");
    List<String> errors = validate(form);

    if (errors.isEmpty()) {
        System.out.println("Form is valid");
    } else {
        for (String error : errors) {
            System.out.println(error);
        }
    }
}
}
```

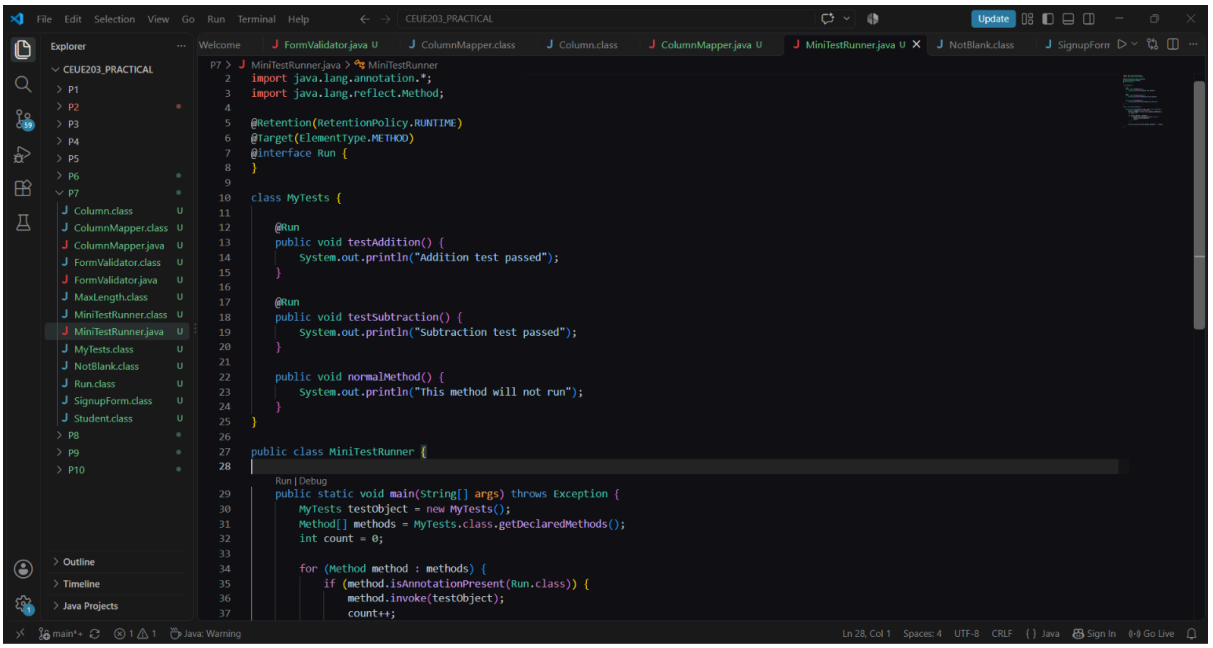
OUTPUT :



```
PS D:\CEUE203_PRACTICAL\P7> cd "d:\CEUE203_PRACTICAL\P7\" ; if ($?) { javac FormValidator.java } ; if ($?) { java FormValidator }
name cannot be blank
PS D:\CEUE203_PRACTICAL\P7> cd "d:\CEUE203_PRACTICAL\P7\" ; if ($?) { javac FormValidator.java } ; if ($?) { java FormValidator }
name exceeds maximum length 20
PS D:\CEUE203_PRACTICAL\P7>
```

2. Mini test runner: define a @Run annotation and mark some no-arg methods of a class with it. Use reflection to find and invoke only the @Run methods and report how many ran — the idea behind how JUnit finds @Test methods.

CODE :



```java
import java.lang.annotation.*;
import java.lang.reflect.Method;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Run {
}

class MyTests {

    @Run
    public void testAddition() {
        System.out.println("Addition test passed");
    }

    @Run
    public void testSubtraction() {
        System.out.println("Subtraction test passed");
    }

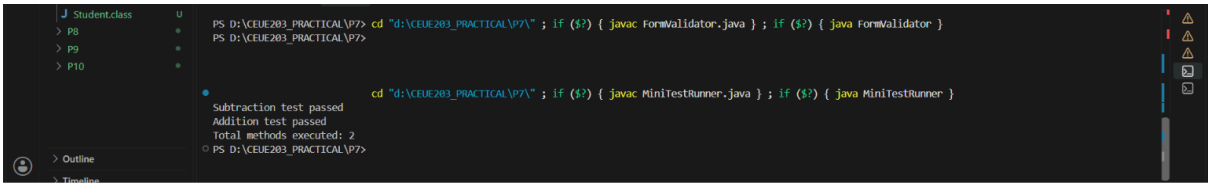
    public void normalMethod() {
        System.out.println("This method will not run");
    }
}

public class MiniTestRunner {

Run | Debug
    public static void main(String[] args) throws Exception {
        MyTests testObject = new MyTests();
        Method[] methods = MyTests.class.getDeclaredMethods();
        int count = 0;

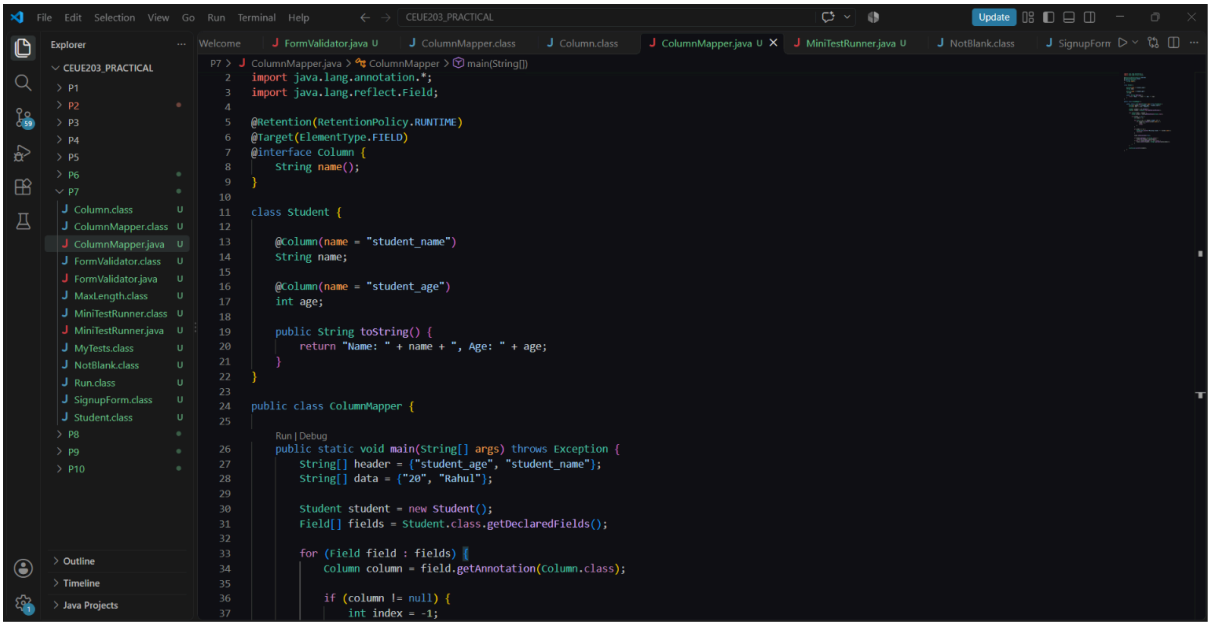
        for (Method method : methods) {
            if (method.isAnnotationPresent(Run.class)) {
                method.invoke(testObject);
                count++;
```

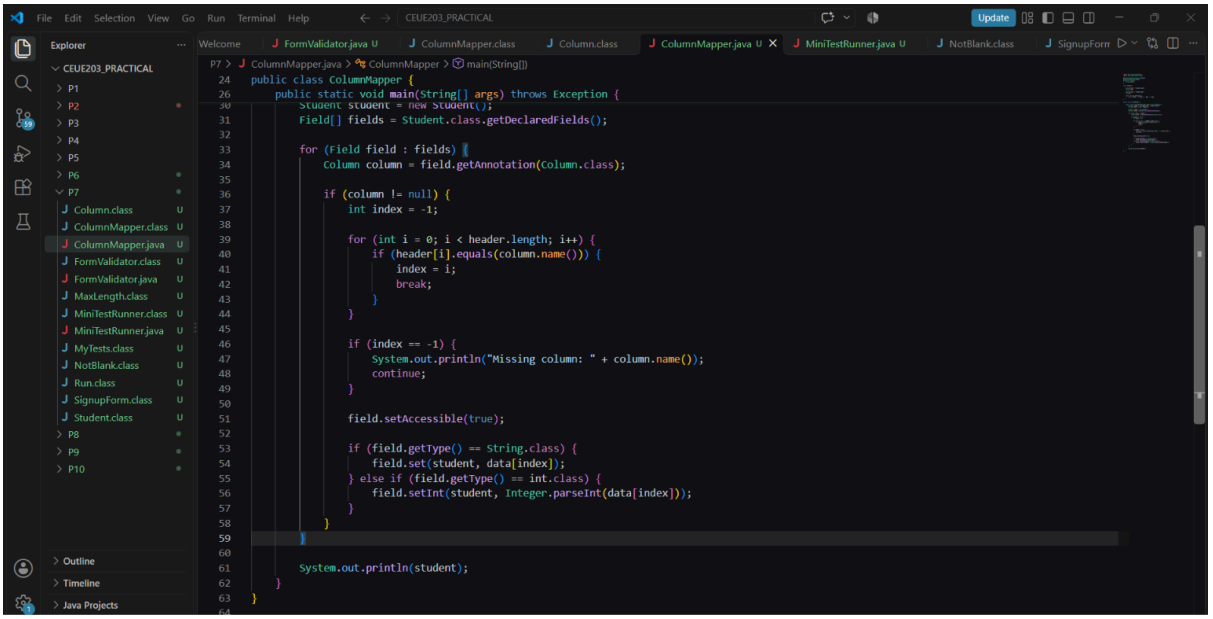
OUTPUT :



3. (For Advanced Learners – home) Define a @Column(name) annotation and, by reflection, populate an object's fields from a header row and a data row by matching column names; decide how to handle a missing column.

CODE :





```java
public class ColumnMapper {
    public static void main(String[] args) throws Exception {
        Student student = new Student();
        Field[] fields = Student.class.getDeclaredFields();

        for (Field field : fields) {
            Column column = field.getAnnotation(Column.class);

            if (column != null) {
                int index = -1;

                for (int i = 0; i < header.length; i++) {
                    if (header[i].equals(column.name())) {
                        index = i;
                        break;
                    }
                }

                if (index == -1) {
                    System.out.println("Missing column: " + column.name());
                    continue;
                }

                field.setAccessible(true);

                if (field.getType() == String.class) {
                    field.set(student, data[index]);
                } else if (field.getType() == int.class) {
                    field.setInt(student, Integer.parseInt(data[index]));
                }
            }
        }

        System.out.println(student);
    }
}
```

OUTPUT :



```
PS D:\CEUE203_PRACTICAL\P7>

Addition test passed
Total methods executed: 2
PS D:\CEUE203_PRACTICAL\P7> cd "d:\CEUE203_PRACTICAL\P7\" ; if ($?) { javac ColumnMapper.java } ; if ($?) { java ColumnMapper }
Name: Rahul, Age: 20
PS D:\CEUE203_PRACTICAL\P7>
```

KEY QUESTIONS :

1. What are the meta-annotations @Retention and @Target used for?

@Retention specifies how long an annotation is available. @Retention(RetentionPolicy.RUNTIME) makes it available at runtime. @Target specifies where an annotation can be used, such as on fields or methods.

2. What is reflection, and what does getDeclaredFields() return?

Reflection is a feature in Java that allows a program to inspect classes, fields, and methods at runtime. getDeclaredFields() returns all fields declared directly in a class, including private fields.

3. How do annotations plus reflection let a framework validate any object?

Annotations define validation rules for fields, such as @NotBlank and @MaxLength. Reflection reads these annotations at runtime and checks the field values according to the rules. The framework then returns a list of validation errors.